09_final_forward_prediction_pipeline

April 28, 2026

1 09 - Final Forward Prediction Pipeline

This notebook is the clean final forward-prediction notebook for the Boom Challenge.

It avoids mixing previous experimental versions and uses the best validated forward configuration:

Target	Model	Feature version	Reason
`P80`	ExtraTrees	v2 fragmentation features	v2 improved fragmentation metrics
`fines_frac`	ExtraTrees	v2 fragmentation features	v2 improved fragmentation metrics
`oversize_frac`	ExtraTrees	v2 fragmentation features	ExtraTrees v2 beat CatBoost in notebook 08b
`R95`	ExtraTrees	v1 distance features	v1 remained better for distance targets
`R50_fines`	ExtraTrees	v1 distance features	v1 remained safer for distance targets
`R50_oversize`	ExtraTrees	v1 distance features	v1 remained better for distance targets

Main outputs:

```
outputs/submissions/prediction_submission.csv
outputs/submissions/prediction_submission_forward_final.csv
outputs/models/final_forward_pipeline.joblib
outputs/models/final_forward_metadata.json
```

1.1 1. Imports and paths

[1]:
```
from pathlib import Path
import json
import warnings
import joblib
import numpy as np
import pandas as pd

from sklearn.base import BaseEstimator, TransformerMixin
```

```
from sklearn.pipeline import Pipeline
from sklearn.model_selection import KFold
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.ensemble import ExtraTreesRegressor

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 180)

PROJECT_ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.
 ↪cwd()
FORWARD_DIR = PROJECT_ROOT / "data" / "raw" / "forward_prediction"
INVERSE_DIR = PROJECT_ROOT / "data" / "raw" / "inverse_design"
OUTPUTS_DIR = PROJECT_ROOT / "outputs"
SUBMISSIONS_DIR = OUTPUTS_DIR / "submissions"
MODELS_DIR = OUTPUTS_DIR / "models"
REPORTS_DIR = PROJECT_ROOT / "reports"

for path in [SUBMISSIONS_DIR, MODELS_DIR, REPORTS_DIR]:
    path.mkdir(parents=True, exist_ok=True)

print("Project root:", PROJECT_ROOT)
print("Forward dir:", FORWARD_DIR)
print("Submissions dir:", SUBMISSIONS_DIR)
print("Models dir:", MODELS_DIR)
```

```
Project root: /home/alouiyaz/projects/boom-challenge-ejecta-prediction
Forward dir: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/data/raw/forward_prediction
Submissions dir: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/outputs/submissions
Models dir: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/outputs/models
```

1.2 2. Load data and constraints

[2]:
```
input_cols = [
    "energy", "angle_rad", "coupling", "strength",
    "porosity", "gravity", "atmosphere", "shape_factor",
]

target_cols = [
    "P80", "fines_frac", "oversize_frac",
    "R95", "R50_fines", "R50_oversize",
]

fragmentation_targets = ["P80", "fines_frac", "oversize_frac"]
```

```
distance_targets = ["R95", "R50_fines", "R50_oversize"]

raw_train = pd.read_csv(FORWARD_DIR / "train.csv")[input_cols]
raw_test = pd.read_csv(FORWARD_DIR / "test.csv")[input_cols]
y = pd.read_csv(FORWARD_DIR / "train_labels.csv")[target_cols]

constraints_path = INVERSE_DIR / "constraints.json"
if constraints_path.exists():
    with open(constraints_path, "r") as f:
        constraints_data = json.load(f)
    output_constraints = constraints_data["constraints"]
else:
    output_constraints = {"p80_min": 96.0, "p80_max": 101.0, "r95_max": 175.0}

p80_min = output_constraints["p80_min"]
p80_max = output_constraints["p80_max"]
r95_max = output_constraints["r95_max"]

print("Raw train:", raw_train.shape)
print("Raw test:", raw_test.shape)
print("Targets:", y.shape)
display(pd.DataFrame([output_constraints]))
display(raw_train.head())
display(y.head())
```

```
Raw train: (2930, 8)
Raw test: (492, 8)
Targets: (2930, 6)
```

	p80_min	p80_max	r95_max
0	96.0	101.0	175.0

	energy	angle_rad	coupling	strength	porosity	gravity	atmosphere	shape_factor
0	3.826405	0.818303	0.861258	1.305809	0.337215	3.71	0.781263	0.784028
1	2.828754	1.193036	0.561245	3.494501	0.058029	1.62	0.136205	0.922737
2	3.068907	0.605872	0.948860	1.366386	0.315632	3.71	0.774704	0.954922
3	2.700574	1.073708	0.713705	3.599419	0.033062	1.62	0.144204	0.932911
4	3.484022	0.863568	1.237205	1.996742	0.278207	9.81	0.414620	1.260855

	P80	fines_frac	oversize_frac	R95	R50_fines	R50_oversize
0	76.972350	0.184728	0.016671	198.938699	175.527939	76.235779
1	269.057465	0.000622	0.916734	239.268477	447.157838	141.894047
2	104.070923	0.070343	0.094438	192.986417	189.286407	84.235774

```
3  257.618403     0.001026        0.880122  289.289693  500.000028     169.866473
4  111.717167     0.058576        0.136166   94.229304   97.614864      42.928393
```

1.3 3. Sklearn-compatible feature engineering

`PhysicsFeatureEngineer(use_advanced=False)` generates the v1 physics features.

`PhysicsFeatureEngineer(use_advanced=True)` adds only the validated v2 advanced features.

[3]:
```python
class PhysicsFeatureEngineer(BaseEstimator, TransformerMixin):
    def __init__(self, use_advanced: bool = False):
        self.use_advanced = use_advanced

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = pd.DataFrame(X).copy()
        X = X[input_cols].copy()
        eps = 1e-9

        # 1. Energy transfer features
        X["effective_energy"] = X["energy"] * X["coupling"]
        X["log_energy"] = np.log1p(X["energy"])
        X["log_effective_energy"] = np.log1p(X["effective_energy"])

        # 2. Angle decomposition
        X["sin_angle"] = np.sin(X["angle_rad"])
        X["cos_angle"] = np.cos(X["angle_rad"])
        X["tan_angle"] = np.tan(X["angle_rad"])
        X["horizontal_energy"] = X["effective_energy"] * X["cos_angle"]
        X["vertical_energy"] = X["effective_energy"] * X["sin_angle"]
        X["vertical_horizontal_ratio"] = X["vertical_energy"] /␣
↪(X["horizontal_energy"] + eps)

        # 3. Material / fragmentation proxies
        X["energy_per_strength"] = X["energy"] / (X["strength"] + eps)
        X["effective_energy_per_strength"] = X["effective_energy"] /␣
↪(X["strength"] + eps)
        X["material_resistance_index"] = X["strength"] * (1 - X["porosity"])
        X["fragmentation_index"] = X["effective_energy"] * X["porosity"] /␣
↪(X["strength"] + eps)
        X["coupling_porosity"] = X["coupling"] * X["porosity"]
        X["coupling_atmosphere"] = X["coupling"] * X["atmosphere"]
        X["porosity_strength"] = X["porosity"] * X["strength"]

        # 4. Gravity and range proxies
        X["energy_per_gravity"] = X["energy"] / (X["gravity"] + eps)
```

```
        X["effective_energy_per_gravity"] = X["effective_energy"] /␣
↪(X["gravity"] + eps)
        X["horizontal_energy_per_gravity"] = X["horizontal_energy"] /␣
↪(X["gravity"] + eps)
        X["vertical_energy_per_gravity"] = X["vertical_energy"] / (X["gravity"]␣
↪+ eps)

        # 5. Atmosphere and drag proxies
        X["drag_proxy"] = X["atmosphere"] * X["shape_factor"]
        X["drag_per_gravity"] = X["drag_proxy"] / (X["gravity"] + eps)
        X["atmosphere_shape_energy"] = X["atmosphere"] * X["shape_factor"] *␣
↪X["effective_energy"]
        X["atmosphere_per_gravity"] = X["atmosphere"] / (X["gravity"] + eps)

        # 6. Pi-like scaling proxies
        X["pi_gravity_proxy"] = (X["gravity"] * X["coupling"]) / (X["energy"] +␣
↪eps)
        X["pi_strength_proxy"] = X["strength"] / (X["gravity"] * X["coupling"]␣
↪+ eps)
        X["pi_atmosphere_proxy"] = X["atmosphere"] / (X["gravity"] *␣
↪X["coupling"] + eps)

        # 7. Regime indicators from EDA
        X["porosity_regime"] = (X["porosity"] > 0.15).astype(int)
        X["strength_regime"] = (X["strength"] > 2.6).astype(int)
        X["angle_regime"] = (X["angle_rad"] > 0.95).astype(int)
        X["atm_regime"] = (X["atmosphere"] > 0.30).astype(int)
        X["regime_combo"] = (
            X["porosity_regime"] * 8
            + X["strength_regime"] * 4
            + X["angle_regime"] * 2
            + X["atm_regime"]
        )

        # 8. Cross-regime proxies
        X["scaled_energy"] = X["effective_energy"] / (X["strength"] * np.
↪sqrt(X["gravity"]) + eps)
        X["fragility"] = X["porosity"] / (X["strength"] + eps)
        X["range_proxy"] = (X["effective_energy"] * (X["cos_angle"] ** 2)) /␣
↪(X["gravity"] * X["strength"] + eps)
        X["energy_sin_angle"] = X["energy"] * X["sin_angle"]
        X["momentum_proxy"] = X["effective_energy"] * X["sin_angle"]
        X["coupling_per_atm_clipped"] = X["coupling"] / (X["atmosphere"] + 1e-3)
        X["log_coupling_per_atm"] = np.log1p(X["coupling_per_atm_clipped"])
        X["retention_factor"] = X["atmosphere"] * X["drag_proxy"] /␣
↪(X["energy"] + eps)
```

```
        # 9. Validated advanced features, used only for fragmentation targets
        if self.use_advanced:
            X["froude_proxy"] = np.sqrt(X["effective_energy"] / (X["gravity"] +␣
 ↪eps))
            X["stress_ratio_compact"] = X["effective_energy"] /␣
 ↪(X["material_resistance_index"] + eps)
            X["sqrt_effective_energy_per_strength"] = np.
 ↪sqrt(X["effective_energy_per_strength"].clip(lower=0))
            X["sqrt_effective_energy_per_gravity"] = np.
 ↪sqrt(X["effective_energy_per_gravity"].clip(lower=0))

        return X


class ColumnSelector(BaseEstimator, TransformerMixin):
    def __init__(self, columns):
        self.columns = list(columns)

    def fit(self, X, y=None):
        missing = [c for c in self.columns if c not in X.columns]
        if missing:
            raise ValueError(f"Missing columns: {missing}")
        return self

    def transform(self, X):
        return X[self.columns].copy()
```

1.4 4. Final selected feature sets

[4]:
```
raw_features = input_cols.copy()

fragmentation_features_v1 = raw_features + [
    "effective_energy", "log_effective_energy",
    "sin_angle", "cos_angle", "horizontal_energy", "vertical_energy",
    "energy_per_strength", "effective_energy_per_strength",
    "material_resistance_index", "fragmentation_index",
    "coupling_porosity", "coupling_atmosphere", "porosity_strength",
    "drag_proxy", "atmosphere_shape_energy",
    "pi_strength_proxy", "pi_atmosphere_proxy", "scaled_energy", "fragility",
    "energy_sin_angle", "momentum_proxy",
    "porosity_regime", "strength_regime", "angle_regime", "atm_regime",␣
 ↪"regime_combo",
]

fragmentation_features_v2 = fragmentation_features_v1 + [
    "stress_ratio_compact",
```

```
    "sqrt_effective_energy_per_strength",
]

distance_features_v1 = raw_features + [
    "effective_energy", "log_effective_energy",
    "sin_angle", "cos_angle", "horizontal_energy", "vertical_energy",
    "energy_per_gravity", "effective_energy_per_gravity",
    "horizontal_energy_per_gravity", "vertical_energy_per_gravity",
    "drag_proxy", "drag_per_gravity", "atmosphere_shape_energy",␣
 ↪"atmosphere_per_gravity",
    "pi_gravity_proxy", "pi_atmosphere_proxy", "scaled_energy", "range_proxy",
    "energy_sin_angle", "momentum_proxy",
    "porosity_regime", "strength_regime", "angle_regime", "atm_regime",␣
 ↪"regime_combo",
]

final_target_config = {
    "P80": {
        "model": "ExtraTrees",
        "feature_version": "v2",
        "features": fragmentation_features_v2,
        "reason": "v2 fragmentation features improved MAE/RMSE/P95",
    },
    "fines_frac": {
        "model": "ExtraTrees",
        "feature_version": "v2",
        "features": fragmentation_features_v2,
        "reason": "v2 fragmentation features improved MAE/RMSE/P95",
    },
    "oversize_frac": {
        "model": "ExtraTrees",
        "feature_version": "v2",
        "features": fragmentation_features_v2,
        "reason": "ExtraTrees v2 beat CatBoost in notebook 08b",
    },
    "R95": {
        "model": "ExtraTrees",
        "feature_version": "v1",
        "features": distance_features_v1,
        "reason": "v1 distance features remained better for R95",
    },
    "R50_fines": {
        "model": "ExtraTrees",
        "feature_version": "v1",
        "features": distance_features_v1,
        "reason": "v1 distance features remained safer for R50_fines",
    },
```

```
    "R50_oversize": {
        "model": "ExtraTrees",
        "feature_version": "v1",
        "features": distance_features_v1,
        "reason": "v1 distance features remained better for R50_oversize",
    },
}

config_summary = pd.DataFrame([
    {
        "target": target,
        "model": cfg["model"],
        "feature_version": cfg["feature_version"],
        "n_features": len(cfg["features"]),
        "reason": cfg["reason"],
    }
    for target, cfg in final_target_config.items()
])
display(config_summary)
```

```
         target       model feature_version  n_features                      ␣
 ↪                    reason
0            P80  ExtraTrees              v2          36    v2 fragmentation␣
 ↪features improved MAE/RMSE/P95
1     fines_frac  ExtraTrees              v2          36    v2 fragmentation␣
 ↪features improved MAE/RMSE/P95
2  oversize_frac  ExtraTrees              v2          36        ExtraTrees v2␣
 ↪beat CatBoost in notebook 08b
3            R95  ExtraTrees              v1          33       v1 distance␣
 ↪features remained better for R95
4      R50_fines  ExtraTrees              v1          33  v1 distance features␣
 ↪remained safer for R50_fines
5   R50_oversize  ExtraTrees              v1          33  v1 distance features␣
 ↪remained better for R50_o…
```

1.5 5. Final pipeline wrapper

[5]:
```
def build_extratrees(random_state=42):
    return ExtraTreesRegressor(
        n_estimators=800,
        max_features="sqrt",
        min_samples_leaf=2,
        random_state=random_state,
        n_jobs=-1,
    )
```

```
def build_target_pipeline(target, random_state=42):
    cfg = final_target_config[target]
    use_advanced = cfg["feature_version"] == "v2"
    return Pipeline(steps=[
        ("features", PhysicsFeatureEngineer(use_advanced=use_advanced)),
        ("select", ColumnSelector(cfg["features"])),
        ("model", build_extratrees(random_state=random_state)),
    ])


def clip_predictions(preds, target):
    preds = np.asarray(preds).copy()
    if target in ["fines_frac", "oversize_frac"]:
        return np.clip(preds, 0, 1)
    return np.clip(preds, 0, None)


class FinalForwardPipeline:
    def __init__(self, target_config, random_state=42):
        self.target_config = target_config
        self.random_state = random_state
        self.pipelines_ = {}

    def fit(self, X_raw, y_df):
        for i, target in enumerate(target_cols):
            pipe = build_target_pipeline(target, random_state=self.random_state␣
 ↪+ i)
            pipe.fit(X_raw, y_df[target])
            self.pipelines_[target] = pipe
        return self

    def predict(self, X_raw):
        preds = pd.DataFrame(index=X_raw.index)
        for target in target_cols:
            pred = self.pipelines_[target].predict(X_raw)
            preds[target] = clip_predictions(pred, target)
        return preds[target_cols]

    def describe(self):
        rows = []
        for target, pipe in self.pipelines_.items():
            cfg = self.target_config[target]
            rows.append({
                "target": target,
                "model": cfg["model"],
                "feature_version": cfg["feature_version"],
                "n_features": len(cfg["features"]),
```

```
                "pipeline_model": type(pipe.named_steps["model"]).__name__,
                "reason": cfg["reason"],
            })
        return pd.DataFrame(rows)
```

1.6 6. Validation metrics

[6]:
```
def regression_metrics(y_true, y_pred, target_name=None):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    abs_error = np.abs(y_pred - y_true)
    out = {
        "MAE": mae,
        "RMSE": rmse,
        "R2": r2,
        "Median_AE": np.median(abs_error),
        "P90_AE": np.percentile(abs_error, 90),
        "P95_AE": np.percentile(abs_error, 95),
        "Max_AE": np.max(abs_error),
        "Bias": float(np.mean(y_pred - y_true)),
    }
    if target_name is not None:
        out["normalized_MAE"] = mae / (y[target_name].std() + 1e-9)
        out["normalized_RMSE"] = rmse / (y[target_name].std() + 1e-9)
    return out


def feasibility_mask(df_targets):
    return (df_targets["P80"].between(p80_min, p80_max)) & (df_targets["R95"]␣
 ↪<= r95_max)


def custom_constraint_metrics(y_true_df, y_pred_df):
    true_feasible = feasibility_mask(y_true_df)
    pred_feasible = feasibility_mask(y_pred_df)

    tp = int((true_feasible & pred_feasible).sum())
    fp = int((~true_feasible & pred_feasible).sum())
    fn = int((true_feasible & ~pred_feasible).sum())

    precision = tp / (tp + fp + 1e-9)
    recall = tp / (tp + fn + 1e-9)
    f1 = 2 * precision * recall / (precision + recall + 1e-9)
```

```
    near_zone = (y_true_df["P80"].between(80, 120)) & (y_true_df["R95"] <= 250)

    out = {
        "n_true_feasible": int(true_feasible.sum()),
        "n_pred_feasible": int(pred_feasible.sum()),
        "true_positive": tp,
        "false_positive": fp,
        "false_negative": fn,
        "feasible_precision": precision,
        "feasible_recall": recall,
        "feasible_f1": f1,
        "near_zone_count": int(near_zone.sum()),
    }

    if near_zone.sum() > 0:
        out["near_zone_MAE_P80"] = mean_absolute_error(
            y_true_df.loc[near_zone, "P80"], y_pred_df.loc[near_zone, "P80"]
        )
        out["near_zone_MAE_R95"] = mean_absolute_error(
            y_true_df.loc[near_zone, "R95"], y_pred_df.loc[near_zone, "R95"]
        )
        out["near_zone_R95_bias"] = float(
            (y_pred_df.loc[near_zone, "R95"] - y_true_df.loc[near_zone, "R95"]).
↪mean()
        )
    return out
```

1.7 7. Optional final cross-validation check

Set `RUN_FINAL_CV = False` if you only want to train the final model and generate the submission quickly.

[7]:
```
def cross_validate_final_model(X_raw, y_df, n_splits=5, random_state=42):
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=random_state)
    oof = pd.DataFrame(index=y_df.index, columns=target_cols, dtype=float)
    fold_rows = []

    for fold, (train_idx, valid_idx) in enumerate(kf.split(X_raw), start=1):
        print(f"Fold {fold}/{n_splits}")
        X_train = X_raw.iloc[train_idx].reset_index(drop=True)
        X_valid = X_raw.iloc[valid_idx].reset_index(drop=True)
        y_train = y_df.iloc[train_idx].reset_index(drop=True)
        y_valid = y_df.iloc[valid_idx].reset_index(drop=True)

        model = FinalForwardPipeline(target_config=final_target_config,␣
↪random_state=random_state + fold)
        model.fit(X_train, y_train)
```

```
        pred_valid = model.predict(X_valid)
        pred_valid.index = valid_idx
        oof.loc[valid_idx, target_cols] = pred_valid[target_cols]

        for target in target_cols:
            metrics = regression_metrics(y_valid[target], pred_valid[target],␣
 ↪target_name=target)
            metrics.update({
                "fold": fold,
                "target": target,
                "feature_version":␣
 ↪final_target_config[target]["feature_version"],
                "n_features": len(final_target_config[target]["features"]),
            })
            fold_rows.append(metrics)

    overall_rows = []
    for target in target_cols:
        metrics = regression_metrics(y_df[target], oof[target],␣
 ↪target_name=target)
        metrics.update({
            "fold": "OOF",
            "target": target,
            "feature_version": final_target_config[target]["feature_version"],
            "n_features": len(final_target_config[target]["features"]),
        })
        overall_rows.append(metrics)

    overall_df = pd.DataFrame(overall_rows)
    fold_df = pd.DataFrame(fold_rows)
    constraint_df = pd.DataFrame([custom_constraint_metrics(y_df, oof)])
    return overall_df, fold_df, oof, constraint_df


RUN_FINAL_CV = True

if RUN_FINAL_CV:
    final_cv_results, final_fold_results, final_oof_predictions,␣
 ↪final_constraint_metrics = cross_validate_final_model(
        raw_train,
        y,
        n_splits=5,
        random_state=42,
    )
    display(final_cv_results.sort_values("normalized_MAE"))
    display(final_constraint_metrics)
else:
```

```
    print("Skipping final CV.")
```

```
Fold 1/5
Fold 2/5
Fold 3/5
Fold 4/5
Fold 5/5
```

```
         MAE       RMSE        R2  Median_AE      P90_AE      P95_AE      Max_AE␣
 ↪     Bias  normalized_MAE  normalized_RMSE fold         target feature_version␣
 ↪ n_features
2   0.025680   0.035189  0.990552   0.018655    0.057401    0.075011    0.167200␣
 ↪-0.000040        0.070923         0.097185  OOF  oversize_frac              v2␣
 ↪         36
1   0.006317   0.013826  0.959368   0.000613    0.019765    0.031609    0.141095␣
 ↪-0.000035        0.092084         0.201540  OOF     fines_frac              v2␣
 ↪         36
0   7.649464  10.159581  0.976120   5.889286   16.804011   21.487673   53.412408␣
 ↪-0.005756        0.116332         0.154506  OOF            P80              v2␣
 ↪         36
3  41.432050  68.507429  0.917493  20.851114  103.401705  149.134690  471.617395␣
 ↪-0.366155        0.173688         0.287191  OOF            R95              v1␣
 ↪         33
4  50.393094  78.496171  0.898887  27.455684  129.306490  170.647824  533.891646␣
 ↪-0.386028        0.204104         0.317929  OOF      R50_fines              v1␣
 ↪         33
5  22.731340  38.575224  0.873407  11.680371   56.905391   81.580273  346.724231␣
 ↪-0.000924        0.209627         0.355739  OOF   R50_oversize              v1␣
 ↪         33
```

```
   n_true_feasible  n_pred_feasible  true_positive  false_positive ␣
 ↪false_negative  feasible_precision  feasible_recall  feasible_f1 ␣
 ↪near_zone_count  near_zone_MAE_P80  \
0               35               37             13              24             ␣
 ↪22            0.351351         0.371429     0.361111             372          ␣
 ↪ 4.584673

   near_zone_MAE_R95  near_zone_R95_bias
0          25.522959           13.911613
```

1.8 8. Train final model on all training data

[8]:
```
final_model = FinalForwardPipeline(target_config=final_target_config,␣
 ↪random_state=42)
final_model.fit(raw_train, y)

display(final_model.describe())
```

```
         target       model feature_version  n_features       pipeline_model␣
↪                                                  reason
0            P80  ExtraTrees              v2          36  ExtraTreesRegressor␣
↪v2 fragmentation features improved MAE/RMSE/P95
1     fines_frac  ExtraTrees              v2          36  ExtraTreesRegressor␣
↪v2 fragmentation features improved MAE/RMSE/P95
2  oversize_frac  ExtraTrees              v2          36  ExtraTreesRegressor␣
↪    ExtraTrees v2 beat CatBoost in notebook 08b
3            R95  ExtraTrees              v1          33  ExtraTreesRegressor␣
↪   v1 distance features remained better for R95
4      R50_fines  ExtraTrees              v1          33  ExtraTreesRegressor␣
↪v1 distance features remained safer for R50_fines
5   R50_oversize  ExtraTrees              v1          33  ExtraTreesRegressor␣
↪v1 distance features remained better for R50_o…
```

1.9 9. Predict test set and create official submission

[9]:
```
test_predictions = final_model.predict(raw_test)

display(test_predictions.head())
display(test_predictions.describe().T)

submission = pd.DataFrame({"scenario_id": np.arange(len(raw_test))})
for col in target_cols:
    submission[col] = test_predictions[col].values
submission = submission[["scenario_id"] + target_cols]

# Official challenge filename.
official_submission_path = SUBMISSIONS_DIR / "prediction_submission.csv"
# Descriptive backup filename.
descriptive_submission_path = SUBMISSIONS_DIR /␣
 ↪"prediction_submission_forward_final.csv"

submission.to_csv(official_submission_path, index=False)
submission.to_csv(descriptive_submission_path, index=False)

print("Saved official submission to:", official_submission_path)
print("Saved descriptive backup to:", descriptive_submission_path)
print("Shape:", submission.shape)
display(submission.head())
display(submission.tail())
```

	P80	fines_frac	oversize_frac	R95	R50_fines	R50_oversize
0	120.852192	0.120046	0.208373	726.196054	704.966598	312.487130
1	113.668032	0.048219	0.158274	204.758075	220.793490	102.212871
2	149.740914	0.121343	0.384067	809.085123	808.873827	348.096898
3	162.227452	0.008732	0.543184	531.683980	592.186388	277.513349
4	137.978607	0.047156	0.357337	197.063393	243.395743	100.912752

```
              count        mean         std        min         25%         50%
↪        75%         max
P80           492.0  157.708072   33.479974  76.016131  130.253486  153.137936
↪182.044124  230.519576
fines_frac    492.0    0.050271    0.059135   0.001579    0.005592    0.017260
↪  0.085643    0.265903
oversize_frac 492.0    0.468492    0.210192   0.033226    0.289992    0.437667
↪  0.648656    0.847572
R95           492.0  298.530443  241.158254  48.776910  109.611058  177.453510
↪508.553148  996.810828
R50_fines     492.0  330.556280  235.993585  64.210862  142.579155  213.098175
↪580.026235  873.066616
R50_oversize  492.0  146.295647  108.429543  28.072745   58.843228   91.697836
↪263.430771  414.330461

Saved official submission to: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/outputs/submissions/prediction_submission.csv
Saved descriptive backup to: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/outputs/submissions/prediction_submission_forward_final.csv
Shape: (492, 7)

   scenario_id         P80  fines_frac  oversize_frac         R95   R50_fines
↪R50_oversize
0            0  120.852192    0.120046       0.208373  726.196054  704.966598
↪312.487130
1            1  113.668032    0.048219       0.158274  204.758075  220.793490
↪102.212871
2            2  149.740914    0.121343       0.384067  809.085123  808.873827
↪348.096898
3            3  162.227452    0.008732       0.543184  531.683980  592.186388
↪277.513349
4            4  137.978607    0.047156       0.357337  197.063393  243.395743
↪100.912752

     scenario_id         P80  fines_frac  oversize_frac         R95   R50_fines
↪R50_oversize
487          487  152.206678    0.085654       0.398060   91.787908  109.365615
↪   44.027042
488          488  137.972752    0.022555       0.374084  581.289278  628.330345
↪  287.685770
489          489  166.438766    0.006598       0.563239   57.146187   70.314348
↪   30.894136
490          490  194.291398    0.003813       0.717706  152.386776  201.155009
↪   86.035565
491          491  207.066582    0.002437       0.774911  372.573623  507.461436
↪  216.989868
```

1.10 10. Basic format validation

[10]:
```
expected_columns = [
    "scenario_id", "P80", "fines_frac", "oversize_frac",
    "R95", "R50_fines", "R50_oversize",
]

assert submission.shape[0] == raw_test.shape[0], "Submission row count does not␣
 ↪match test set."
assert submission.columns.tolist() == expected_columns, "Submission columns are␣
 ↪not in the required order."
assert submission["scenario_id"].tolist() == list(range(len(raw_test))),␣
 ↪"scenario_id must be 0-based row index."
assert np.isfinite(submission[target_cols].values).all(), "Submission contains␣
 ↪non-finite predictions."
assert (submission[["P80", "R95", "R50_fines", "R50_oversize"]] >= 0).all().
 ↪all(), "Distance/size predictions must be non-negative."
assert ((submission[["fines_frac", "oversize_frac"]] >= 0) &␣
 ↪(submission[["fines_frac", "oversize_frac"]] <= 1)).all().all(), "Fraction␣
 ↪predictions must be in [0, 1]."

print("Submission format validation passed.")
```

```
Submission format validation passed.
```

1.11 11. Compare final submission with previous files, if available

[11]:
```
comparison_files = {
    "v1_target_specific": SUBMISSIONS_DIR /␣
 ↪"prediction_submission_final_target_specific.csv",
    "v2_hybrid_extratrees": SUBMISSIONS_DIR /␣
 ↪"prediction_submission_forward_v2_hybrid.csv",
    "v2_best_models": SUBMISSIONS_DIR /␣
 ↪"prediction_submission_forward_v2_best_models.csv",
    "final_selected": official_submission_path,
}

loaded = {}
for name, path in comparison_files.items():
    if path.exists():
        loaded[name] = pd.read_csv(path)
        print(name, path, loaded[name].shape)
    else:
        print("Missing", name, path)

rows = []
for name, df in loaded.items():
```

```
    for target in target_cols:
        rows.append({
            "file": name,
            "target": target,
            "mean": df[target].mean(),
            "std": df[target].std(),
            "min": df[target].min(),
            "median": df[target].median(),
            "max": df[target].max(),
        })

test_distribution_comparison = pd.DataFrame(rows)
display(test_distribution_comparison)

if "v1_target_specific" in loaded:
    diff_rows = []
    v1 = loaded["v1_target_specific"]
    final = loaded["final_selected"]
    for target in target_cols:
        diff = final[target] - v1[target]
        diff_rows.append({
            "target": target,
            "mean_diff_final_minus_v1": diff.mean(),
            "median_abs_diff": diff.abs().median(),
            "max_abs_diff": diff.abs().max(),
        })
    test_prediction_diff_vs_v1 = pd.DataFrame(diff_rows)
    display(test_prediction_diff_vs_v1)
```

```
v1_target_specific /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/outputs/submissions/prediction_submission_final_target_specific.csv
(492, 7)
v2_hybrid_extratrees /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/outputs/submissions/prediction_submission_forward_v2_hybrid.csv (492,
7)
v2_best_models /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/outputs/submissions/prediction_submission_forward_v2_best_models.csv
(492, 7)
final_selected /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/outputs/submissions/prediction_submission.csv (492, 7)
```

```
                  file         target        mean         std        min      ␣
 ↪median         max
0     v1_target_specific            P80  154.410581   29.205843  77.932694  152.
 ↪472220  219.854089
1     v1_target_specific     fines_frac    0.047445    0.051733   0.002168    0.
 ↪019887    0.240072
```

```
2     v1_target_specific  oversize_frac    0.432358    0.224458   0.045259    0.
↪416259    0.801378
3     v1_target_specific           R95  288.544250  230.479605  48.541924  171.
↪983077  945.481917
4     v1_target_specific      R50_fines  323.398563  226.841124  63.706866  207.
↪213529  825.765013
5     v1_target_specific   R50_oversize  142.488865  105.015312  27.498061   88.
↪196924  406.201873
6   v2_hybrid_extratrees           P80  157.708072   33.479974  76.016131  153.
↪137936  230.519576
7   v2_hybrid_extratrees     fines_frac    0.050271    0.059135   0.001579    0.
↪017260    0.265903
8   v2_hybrid_extratrees  oversize_frac    0.468492    0.210192   0.033226    0.
↪437667    0.847572
9   v2_hybrid_extratrees           R95  298.530443  241.158254  48.776910  177.
↪453510  996.810828
10  v2_hybrid_extratrees      R50_fines  330.556280  235.993585  64.210862  213.
↪098175  873.066616
11  v2_hybrid_extratrees   R50_oversize  146.295647  108.429543  28.072745   91.
↪697836  414.330461
12        v2_best_models           P80  157.708072   33.479974  76.016131  153.
↪137936  230.519576
13        v2_best_models     fines_frac    0.050271    0.059135   0.001579    0.
↪017260    0.265903
14        v2_best_models  oversize_frac    0.422195    0.238924   0.030346    0.
↪400134    0.796151
15        v2_best_models           R95  298.530443  241.158254  48.776910  177.
↪453510  996.810828
16        v2_best_models      R50_fines  330.556280  235.993585  64.210862  213.
↪098175  873.066616
17        v2_best_models   R50_oversize  146.295647  108.429543  28.072745   91.
↪697836  414.330461
18        final_selected           P80  157.708072   33.479974  76.016131  153.
↪137936  230.519576
19        final_selected     fines_frac    0.050271    0.059135   0.001579    0.
↪017260    0.265903
20        final_selected  oversize_frac    0.468492    0.210192   0.033226    0.
↪437667    0.847572
21        final_selected           R95  298.530443  241.158254  48.776910  177.
↪453510  996.810828
22        final_selected      R50_fines  330.556280  235.993585  64.210862  213.
↪098175  873.066616
23        final_selected   R50_oversize  146.295647  108.429543  28.072745   91.
↪697836  414.330461

          target  mean_diff_final_minus_v1  median_abs_diff  max_abs_diff
```

0	P80	3.297491	3.365202	13.683074
1	fines_frac	0.002826	0.002045	0.044400
2	oversize_frac	0.036135	0.041852	0.246055
3	R95	9.986193	7.163189	66.517917
4	R50_fines	7.157717	5.235717	47.301603
5	R50_oversize	3.806782	2.440697	27.651072

1.12 12. Save model and metadata

[12]:
```
model_path = MODELS_DIR / "final_forward_pipeline.joblib"
joblib.dump(final_model, model_path)

metadata = {
    "model_name": "FinalForwardPipeline",
    "target_columns": target_cols,
    "input_columns": input_cols,
    "target_config": {
        target: {
            "model": cfg["model"],
            "feature_version": cfg["feature_version"],
            "n_features": len(cfg["features"]),
            "features": cfg["features"],
            "reason": cfg["reason"],
        }
        for target, cfg in final_target_config.items()
    },
    "official_submission_path": str(official_submission_path),
    "descriptive_submission_path": str(descriptive_submission_path),
    "model_path": str(model_path),
    "constraints": output_constraints,
}

metadata_path = MODELS_DIR / "final_forward_metadata.json"
with open(metadata_path, "w") as f:
    json.dump(metadata, f, indent=2)

print("Saved model to:", model_path)
print("Saved metadata to:", metadata_path)
```

```
Saved model to: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/outputs/models/final_forward_pipeline.joblib
Saved metadata to: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/outputs/models/final_forward_metadata.json
```

1.13 Final decision

Use this file for the forward-prediction submission:

`outputs/submissions/prediction_submission.csv`

This notebook is the single clean forward final pipeline. Earlier notebooks remain useful as experiments, but this notebook should be treated as the forward-prediction source of truth.